ALIVE

Adaptive Life in Virtual Environments

Learning Within Lifetimes, Evolving Across Generations

A controlled study of learning, inherited adaptation, and an expandable modular trait graph

Two-semester master's project | Revised September 2, 2026

Original proposal prepared by Akshay Kumar. Revised for the ALIVE team following advisor-style review.

Abstract

ALIVE will investigate how learning during an individual lifetime interacts with inherited adaptation across generations. Digital agents will compete for finite, regrowing resources while researchers control the frequency of environmental change. A four-condition comparison will enable or disable lifetime learning and cumulative inherited adaptation, with stable-world controls and paired random seeds. The initial system will use eight ability modules, four continuous traits, and a small learner that updates preferences for exploration, harvesting, contesting, and conservation. Primary outcomes will measure sustained population performance and inherited module diversity. Standardized tests of archived designs will separate inherited improvement from benefits that require lifetime experience. The study extends established work on learning and evolution by examining these mechanisms within explicit modular constraints; a matched prerequisite-removal comparison will test the contribution of those constraints. Development will begin with an early complete pilot and expand through validation gates toward a larger trait graph. The intended result is a reproducible account of learning-evolution interactions within the tested worlds and architectures, including null effects, trade-offs, and extinction.

Central question How does the speed of environmental change affect the interaction between lifetime learning and inherited adaptation in agents with modular abilities?

1. Research contribution and questions

Learning can guide evolution without learned behavior being directly inherited. Hinton and Nowlan demonstrated this interaction in 1987 [1]. Aguilar, Bennati, and Helbing later studied foraging agents in cyclically changing environments and showed that learning can alter which inherited strategies evolve [2]. ALIVE therefore treats the interaction itself as established research territory.

The proposed contribution is a controlled extension focused on modular abilities: prerequisites, energy costs, and restricted combinations may change how learning affects performance and persistent inherited diversity. This is a candidate contribution to substantiate through a focused literature comparison before design lock. A larger simulator or a larger trait tree alone does not establish scientific novelty.

Primary questions

- Does the effect of combining learning and inherited adaptation differ between slowly and rapidly changing resource worlds?
- Does learning preserve several persistent module configurations, or favor a narrower set of flexible inherited designs?

Mechanism and architecture questions

- Do later inherited designs perform better from birth under standardized conditions, and how much of their performance still depends on learning?
- Does removing prerequisite edges, while preserving module functions and costs, change the measured interaction?

Predictions are conditional. Learning may improve short-term response when information remains useful long enough to act on it. Costs, misleading feedback, or frequent resets may remove that advantage. Evolution may favor specialization during long stable periods or flexible designs under recurrent change. The combined condition is not assumed to win.

Scope of inference

The core experiment estimates effects for the implemented learner, graph, resource rules, and schedules. It does not establish biological realism, human behavior, unlimited trait invention, or a universal advantage of learning. The project retains the synthetic-agent framing; language-model and digital-twin studies are background inspiration rather than evidence for its research gap.

Minimum scientific result Estimate the learning-by-inherited-adaptation interaction with uncertainty, and test whether archived designs support the proposed explanation.

2. Simulator and lifetime learning

A fixed-capacity JAX grid simulation will track agent position, energy, age, inherited design, and lifetime state. Resources regrow under researcher-defined rules. Movement, harvesting, metabolic costs, automatic energy-gated reproduction, and death use explicit event accounting. Newborn memory resets, and learned preference values are never inherited in the core study.

Two labeled populations remain for continuity with ALIVE. Both use the same rules and experimental setting; labels do not confer abilities or favoritism. They are reporting groups within one interacting system, not independent replicates or evidence of reciprocal coevolution. A one-population smoke test will check that labels do not change behavior.

Small, explicit learner

Each agent stores four values Q: explore, harvest, contest, and conserve. A common fixed controller translates a selected mode into actions using local observations, energy, and active modules. All conditions use the same controller and action-availability rules. Learning-disabled agents retain the initial Q0 values; they can still react to observations.

At each mode decision, select an available mode using an epsilon mixture of uniform exploration and softmax(Q / temperature). Hold the mode for H steps unless death or an invalid action interrupts it. Accumulate reward over the executed interval, then apply:

```
Q_decay = (1 - decay) * Q + decay * Q0
Q_next[mode] = Q_decay[mode] + alpha * (reward - Q_decay[mode])
```

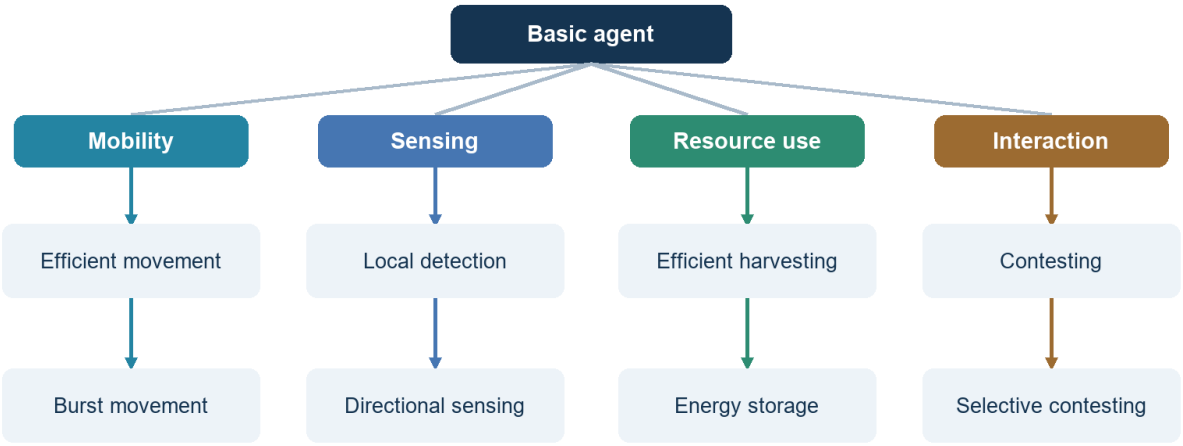
Other entries retain Q_decay. Reward is the bounded, normalized net energy flow from resource acquisition and operating costs over the interval. A separate event ledger excludes reproductive transfers and avoids treating energy-cap clipping as failure. Death receives a fixed terminal penalty. Normalization, H, penalty, exploration, and all update parameters are fixed after pilot testing.

A fixed learning maintenance cost is deducted once per simulation step for learning-enabled agents and included once in operating costs. Learning rate, decay, cost, temperature, and Q0 do not evolve in the core study. All agents reset Q at birth. A short mode horizon only partly addresses delayed benefits; conclusions remain specific to this learner.

Required learner checks Demonstrate successful harvesting, beneficial exploration with delayed payoff, switching after resource relocation, and correct reproduction/death accounting before population experiments. If these fail, revise and retest the learner during the pilot.

3. Trait graph: small core, large-tree roadmap

The first validated model will contain eight modules and four continuous traits: scale, efficiency, sensing, and storage. All agents retain basic movement and harvesting. Modules add or modify capabilities; continuous traits change their magnitude only where the relevant capability exists. Every benefit carries an explicit cost or trade-off.



Illustrative version 1: four family labels, eight functional modules. Downward arrows specify prerequisites. Final effects and costs are validated during the pilot.

Stage	Target	Entry condition
Validated core	8 modules; 4 traits	Complete all four conditions and validate learner, inheritance, costs, and metrics.
Expanded ALIVE	20-30 modules; up to 8 traits	Core analysis complete; add branches in small, separately validated batches.
Large trait tree	40-60+ modules; deeper branches	Long-term target, contingent on evidence, throughput, and team capacity.

Expanded branches may include protection, recovery, and flexibility. Those continuous traits, plus intensity, restore the original eight-trait ambition. The graph can later support cross-branch prerequisites while remaining acyclic. These are planned research extensions, not guaranteed thesis deliverables.

4. Four conditions and inheritance controls

Condition	Lifetime update	Newborn inherited design
Fixed-reference agents	Off; Q = Q0	Drawn from fixed initial distribution P0
Learning only	On; resets at birth	Drawn from the same fixed P0
Inherited adaptation only	Off; Q = Q0	Copied from successful parent, with mutation
Learning + inherited adaptation	On; resets at birth	Copied from successful parent, with mutation

What the inheritance switch means

In all conditions, reproduction requires the same energy threshold and charges the same parental energy cost. Offspring receive the same starting energy and placement rule. In reference conditions, offspring genotype is sampled independently from P0 instead of copied from the parent. This disables cumulative parent-to-offspring genetic adaptation while preserving ecological births and deaths.

Differential survival can still change the traits represented among living reference agents. The design therefore calls these fixed-reference birth distributions, not completely fixed living populations. Birth and survivor distributions are logged separately. Disabling mutation alone is not a no-adaptation control because selection can act on standing inherited variation.

The inherited-adaptation effect includes inheritance, selection across generations, and mutation as a package. It is not interpreted as the isolated effect of mutation. A required small sensitivity study will retain parental inheritance but disable mutation in learning-on and learning-off runs to distinguish selection on initial variation from the addition of new variants.

Mutation and accounting

Mutation makes either a bounded continuous-trait change or one valid module addition/removal. Removing a prerequisite with active dependents is rejected; invalid proposals leave the genotype unchanged. Mutation rates, boundary handling, and rejection rates are recorded. Dead slots reset all lifetime state before reuse. Full-capacity reproduction uses one predetermined allocation rule, and saturation is logged.

Fair comparisons

Conditions share starting samples and precomputed external resource schedules. Separate random streams handle world forcing, action choice, births, and mutation so different event counts do not shift environmental randomness. Resource stocks may diverge through consumption; identical external rules do not imply identical realized worlds. Both labeled populations receive the same treatment.

5. Environmental design and replication

The primary worlds use two regimes: broad abundance and patchy scarcity. The same regime equations are used in stable and changing treatments. Moving resource opportunities remain an extension after the main result. Returning to the original regime is built into repeated cycles, allowing recovery to be measured repeatedly.

World treatment	Protocol	Purpose
Stable abundance	Constant broad resources	Control for elapsed time in an easy world
Stable scarcity	Constant patchy resources	Control for elapsed time in a difficult world
Slow change	Repeated abundance/scarcity cycles	Change slower than baseline turnover
Fast change	The same regimes, shorter periods	Change faster than baseline turnover

Estimate baseline lifetime and turnover during calibration using the reference ecology. Choose slow and fast durations relative to that common benchmark; report realized lifetimes for every condition, but do not alter schedules separately by condition. Pilot targets are roughly 0.5-1 baseline lifetime per fast regime and 5-10 per slow regime, subject to feasibility.

All treatments run for the same total number of steps. Choose durations so changing runs contain complete cycles and equal time in each regime. Balance starting regime across seed blocks. Agents do not observe a global regime label or future schedule. Lock durations and any transition smoothing before confirmatory runs.

Run plan

Cross four adaptation conditions with four world treatments. Target 24 seed blocks, paired across all 16 cells: 384 primary runs. A benchmark-supported range of 20-30 blocks gives 320-480 runs; finalize the number before confirmatory data collection using pilot variability and a stated precision target. Agents, time steps, and population labels are not independent replicates.

Required focused comparisons

- Architecture: repeat all four adaptation conditions in the slow and fast worlds with prerequisite edges removed. Keep module effects, costs, mutation attempts, and initial common-valid designs matched. Interpret changed reachability as part of this intervention; report acceptance rates.
- Control sensitivity: in slow and fast worlds, test parental inheritance without mutation for learning on and off. Use the same preselected subset of 8-12 paired seeds for these focused comparisons; report their lower precision.

With 8-12 blocks, architecture and control sensitivity add 64-96 and 32-48 runs respectively. Learning-cost sweeps and moving-resource worlds follow only if the core and required checks are complete.

6. Measurements and mechanism tests

Two primary outcomes

- Population performance: mean living-agent count over a fixed horizon, divided by initial agent count. This measures sustained population presence, includes extinction, and uses a common denominator. Report pool saturation to detect ceiling effects.
- Inherited module diversity: time-averaged effective number of module configurations, exp(-sum(p * log(p))), where p is each configuration's frequency among living agents at a checkpoint. Extinction contributes zero; also report diversity conditional on survival and population size.

Use module bitmasks for the primary diversity measure. Continuous-trait spread, persistent configurations, action diversity, reproduction, resource efficiency, and energy balance are secondary. Define persistence as frequency above a fixed threshold over a baseline-turnover window; freeze that threshold before main runs.

Separate inheritance from experience

Archive genotypes at predetermined checkpoints across all seed blocks. Evaluate early and late designs as newborn agents with equal age, energy, reset Q, and standardized resource worlds. Use equal sample sizes and a fixed reference competitor cohort. Disable reproduction and mutation during these short evaluations; record survival and resource acquisition.

For each design, run learning enabled and disabled. Better late-design performance without learning supports inherited improvement. The difference between learning-on and learning-off performance measures dependence on learning. Smaller Q changes alone are insufficient evidence. Evaluate the same genotypes in both resource regimes and compare evolved designs from both learning histories.

Recovery and statistical analysis

Define adaptation lag relative to a preregistered fraction of the destination-regime stable-control performance, not the abundance performance before a scarcity shock. Recovery must persist for a fixed window. Runs that never recover are censored; extinction remains a reported outcome. Shock loss integrates the positive deficit below the same reference. Match elapsed time when comparing return cycles with stable controls.

Estimate the interaction on the preregistered outcome scale: combined - learning-only - inherited-only + fixed-reference. Compute paired contrasts by seed block and report 95% confidence intervals using block resampling. A repeated-measures model with seed/run clustering can support period-level analysis. A negative interaction is not by itself proof of reduced selection; use archived-design tests and lineage evidence.

Preregister primary contrasts across slow and fast schedules, multiplicity handling for the two outcomes, exclusion rules limited to verified implementation failures, and extinction handling. Report all seed outcomes. Sensitivity analyses supplement the registered definitions rather than selecting the most favorable one.

7. Two-semester implementation plan

Period	Work	Required milestone
Semester 1 Weeks 1-2	Literature and minimal equations	Closest-work comparison, control definitions, resource and reward accounting
Weeks 3-6	Small complete pilot	All four adaptation conditions; 8-module graph; learner and inheritance tests
Weeks 7-10	Validate and benchmark	CPU/GPU throughput, stable ecology, memory reset, event ledger, synthetic metric traces
Weeks 11-13	Calibrate study	Slow/fast regimes, reference-process sensitivity, parameter robustness, precision planning
Weeks 14-16	Freeze version 1	Preregister graph, learner, endpoints, schedules, seeds, compute and storage budget
Semester 2 Weeks 1-5	Primary experiments	Complete paired matrix, checkpoints and preliminary registered analysis
Weeks 6-8	Required mechanism checks	Archived-genotype tests, prerequisite removal, mutation-off sensitivity
Weeks 9-11	Conditional expansion	Validate selected new branches if core evidence and budget permit
Weeks 12-16	Analysis and writing	Final uncertainty estimates, limitations, thesis, reproducible release

Expansion gates

- Before 20-30 modules: version 1 runs reliably, learner tests pass, primary analysis is complete, and at least one explanation has been tested. A null result qualifies; a desired positive effect is not required.
- Add 2-4 modules at a time. Each batch needs a demonstrated function, a measurable cost, valid inheritance, and no unintended effect on old configurations. Rebenchmark and test on pilot seeds.
- Give each graph version immutable module IDs, effect definitions, prerequisite edges, cost tables, and compatibility tests. Store graph version in every result. Recompile fixed JAX shapes between versions when needed; do not change dimensions within a run.
- Large-tree experiments are a new registered study with their own controls and seeds. Compare conclusions across graph versions; do not merge their raw diversity counts or reuse the original version's claim without retesting.

8. Compute, risks, and delivery

Budget based on measured throughput

The earlier 100-250 GPU-hour figure is retained only as a provisional planning envelope. It is not yet a measured requirement, and the larger trait tree has a separate budget. The core learner has no per-agent neural training loop; GPU need will depend on grid size, interaction rules, agent capacity, horizon, and batching.

Benchmark the full simulator with learning, mutation, and realistic logging by semester 1, week 10. Report compilation time separately from warmed execution, along with hardware, grid size, active/capacity counts, memory use, and wall time per run. Compare CPU and GPU using identical workloads; small runs can remain on CPU.

```
budget hours = run count * measured seconds per run / 3600
```

The required simulation matrix contains approximately 416-624 runs before archived-design evaluations, calibration, and reruns. As arithmetic examples only, 10 minutes per run implies about 69-104 hours; 30 minutes implies 208-312 hours. Add separately measured evaluation and development costs plus a 20% contingency. Monetary cost equals final GPU-hours times the team's actual rate; no price is assumed.

Main risks and responses

- Calibration bias: use pilot seeds to establish viable dynamics and functional mechanisms, not to make the combined condition win. Freeze the main design and use unseen confirmatory seeds.
- Learning or graph artifacts: retain the reward ledger, simple learner tests, matched prerequisite comparison, and archive evaluations. Interpret reduced learning, convergence, or collapse without changing the main world afterward.
- Unreliable timing or excessive scope: reduce graph expansion and optional sweeps first. Preserve the four-condition core and required controls; revise replication only before confirmatory runs and disclose the change.
- Storage bottlenecks: aggregate summaries on device, export at fixed checkpoints, and sample detailed agent traces. Save complete lineage events in batches where feasible and state any sampling limits.

Deliverables and completion criteria

Deliver the tested simulator, immutable version 1 graph, experiment configurations and seeds, throughput report, registered analysis, archived-design evaluations, uncertainty estimates, thesis, and reproducible figures. Deliver a documented expansion interface and large-tree roadmap even if the expanded graph is not built during the thesis. Scientific completion depends on valid estimates and honest interpretation, not a particular condition winning.

9. Revision record and references

Changes from the team proposal

- Preserved ALIVE, the team's learning/evolution question, modular abilities, and researcher-controlled environments. The long-term ambition remains a large trait tree.
- Replaced the initial 20-30-module commitment with a validated eight-module core and explicit expansion gates.
- Narrowed the contribution relative to existing learning-and-evolution research; added a direct prerequisite-removal comparison.
- Clarified the fixed-reference birth process, residual survival selection, and the combined meaning of the inheritance switch.
- Specified a candidate learner with reward accounting and validation tests; held learning parameters fixed initially.
- Made stable controls, slow/fast environmental change, archived-genotype evaluations, and paired run-level inference part of the required study.
- Moved the complete pilot to weeks 3-6 and replaced an unsupported compute estimate with benchmark-based budgeting.

Selected research references

[1] Hinton, G. E., & Nowlan, S. J. (1987). How Learning Can Guide Evolution. Complex Systems, 1, 495-502.

https://www.cs.toronto.edu/~hinton/absps/evolution.htm

[2] Aguilar, L., Bennati, S., & Helbing, D. (2019). How learning can change the course of evolution. PLOS ONE, 14(9), e0219502.

https://doi.org/10.1371/journal.pone.0219502

[3] Tang, Y., Tian, Y., & Ha, D. (2022). EvoJAX: Hardware-Accelerated Neuroevolution. arXiv:2202.05008. Engineering reference for accelerated evolutionary workloads; not a benchmark for this proposed simulator.

https://arxiv.org/abs/2202.05008

Design status

This is a revised proposal, not a record of completed experiments. Module effects, numerical world settings, learner parameters, and final replication count will be fixed through the stated pilot and preregistration process. The proposed 40-60+ module target expresses the team's longer-term direction and is not a prerequisite for a successful thesis.